



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Grado en Ingeniería Informática

Diseño e implementación de un ingeniero de carrera virtual para el simulador F1 25 mediante telemetría UDP, simulación Monte Carlo y modelos de lenguaje

Trabajo fin de estudio presentado por:	Víctor Martín Álvarez
Director/a:	Thiago Ferreira de Toledo
Fecha:	8 Abril 2026
Repositorio del código fuente:	[placeholder]

Resumen

Este Trabajo Fin de Grado aborda el diseño e implementación de un sistema de asistencia estratégica para pilotos del simulador EA Sports F1 25, capaz de operar en tiempo real durante una carrera y emular las funciones de un ingeniero de carrera profesional. La solución se articula en seis componentes acoplados: un servicio de ingesta de telemetría UDP a 20 Hz en Java sobre Oracle AI Database 26ai, un módulo de calibración estadística que ajusta empíricamente los parámetros físicos del simulador, un motor de simulación Monte Carlo que produce distribuciones de probabilidad de posición final bajo diferentes estrategias de parada en boxes, una capa de coordinación mediante Oracle TxEventQ, un portal web Angular para el seguimiento de la carrera y un cliente nativo iOS que entrega los mensajes al piloto por síntesis de voz. La calibración separa los regímenes físicos del coche del jugador y de los oponentes controlados por la IA del juego, hipótesis cuya validación empírica se aborda sobre los datos recogidos. La memoria documenta el diseño arquitectónico, la implementación de los seis componentes y el protocolo experimental para evaluar latencia extremo a extremo, convergencia del motor Monte Carlo, ajuste de la calibración y calidad cualitativa de los mensajes de voz. Esta entrega corresponde al primer hito del proyecto y fija la base sobre la que se completará la caracterización experimental cuantitativa.

Palabras clave: Fórmula 1, telemetría UDP, simulación Monte Carlo, Oracle AI Database 26ai, modelos de lenguaje.

Abstract

This Final Degree Project addresses the design and implementation of a strategic assistance system for drivers of the EA Sports F1 25 simulator, capable of operating in real time and emulating the functions of a professional race engineer. The solution is structured around six coupled components: a UDP telemetry ingestion service running at 20 Hz, developed in Java on top of Oracle AI Database 26ai; a statistical calibration module that empirically fits the simulator's physical parameters from accumulated real-world data; a Monte Carlo simulation engine that produces probability distributions of final positions under different pit stop strategies; a coordination layer based on Oracle TxEventQ transactional queues; an Angular web portal for live monitoring; and a native iOS client that delivers messages to the driver through speech synthesis. The calibration separates the physical regime of the player's car from that of the opponents controlled by the game's artificial intelligence, a hypothesis whose validity is examined over the collected data. Experimental validation covers four axes: pipeline latency, Monte Carlo convergence, statistical fit of the calibration model, and qualitative plausibility of the generated voice messages. This deliverable is a first checkpoint of the work: the architecture is implemented and operational, while the experimental results are completed in the final delivery.

Keywords: Formula 1, UDP telemetry, Monte Carlo simulation, Oracle AI Database 26ai, large language models.

Índice de contenidos

1.	Introducción.....	1
1.1.	Motivación	1
1.2.	Planteamiento del trabajo.....	4
1.3.	Estructura del trabajo.....	6
2.	Contexto y Estado del Arte	8
2.1.	Estrategia de carrera en Fórmula 1	8
2.2.	Simulación académica de carreras	8
2.3.	Sim racing como entorno de investigación	9
2.4.	Modelos de lenguaje como capa de interacción.....	10
2.5.	Síntesis y posicionamiento	11
3.	Objetivos y metodología de trabajo	12
3.1.	Objetivo general.....	12
3.2.	Objetivos específicos.....	12
3.3.	Metodología de trabajo	14
3.3.1.	Fase de análisis del problema y revisión bibliográfica.....	14
3.3.2.	Fase de diseño arquitectónico	14
3.3.3.	Fase de implementación incremental	15
3.3.4.	Fase de calibración y ajuste empírico.....	15
3.3.5.	Fase de evaluación experimental	15
3.3.6.	Herramientas y entornos de desarrollo	16
4.	Título del capítulo	17
4.1.	Visión general del sistema.....	17
4.2.	Decisiones arquitectónicas fundamentales.....	18
4.2.1.	Java sin frameworks para el componente de ingesta	18

4.2.2.	Acceso a base de datos mediante JDBC frente a ORMs	18
4.2.3.	Granularidad por sector como unidad temporal.....	19
4.2.4.	Calibración dual: regímenes físicos del jugador y de la IA del juego	19
4.2.5.	Coordinación entre componentes mediante colas transaccionales.....	20
4.2.6.	Desnormalización selectiva del esquema relacional	20
4.3.	Modelado de datos	21
4.4.	Flujos de información	22
4.4.1.	Flujo de ingesta y persistencia	22
4.4.2.	Flujo de calibración	23
4.4.3.	Flujo de simulación.....	24
4.4.4.	Flujo de salida hacia portal y cliente iOS	24
4.5.	Consideraciones sobre seguridad y gestión del estado	24
5.	Conclusiones y trabajo futuro.....	26
5.1.	Servicio de ingesta de telemetría	26
5.1.1.	Bucle principal y gestión del socket UDP.....	26
5.1.2.	Decodificación de los paquetes.....	26
5.1.3.	Detección de sectores completados y persistencia de instantáneas	26
5.1.4.	Estrategia de recuperación frente a pérdida de paquetes UDP	27
5.2.	Esquema de la base de datos y procedimientos de persistencia	27
5.3.	Módulo de calibración estadística.....	28
5.3.1.	Modelo matemático del tiempo por sector	28
5.3.2.	Procedimiento de ajuste y separación de regímenes	29
5.3.3.	Detección de valores atípicos.....	30
5.3.4.	Persistencia y versionado de los coeficientes	30
5.4.	Motor de simulación Monte Carlo	30

5.4.1.	Estructura interna de una iteración	30
5.4.2.	Generación y poda de estrategias candidatas.....	31
5.4.3.	Agregación de resultados y producción de la respuesta	31
5.5.	Capa de coordinación mediante TxEventQ	32
5.6.	Backend, portal web y cliente iOS	32
5.7.	Capa de generación de mensajes mediante modelo de lenguaje.....	33
6.	Evaluación y resultados	35
6.1.	Latencia extremo a extremo del pipeline.....	35
6.2.	Convergencia del motor de simulación Monte Carlo	35
6.3.	Ajuste estadístico del modelo de calibración	35
6.4.	Valoración cualitativa de los mensajes de voz generados	35
6.5.	Discusión global de los resultados.....	35
7.	Conclusiones y trabajo futuro	36
7.1.	Conclusiones del trabajo	36
7.2.	Lineas de trabajo futuro	36
	Referencias bibliográficas	37
	Anexo A. Título del anexo.....	39
	Índice de acrónimos	40

Índice de figuras

Figura 1. Visión general del sistema y flujos principales entre componentes.	6
---	---

Índice de tablas

Tabla 1. Usuarios potenciales de un documento de requisitos.	3
---	---

1. Introducción

[Placeholder]

1.1. Motivación

La estrategia de carrera constituye uno de los factores más determinantes en el resultado final de una competición de Fórmula 1. La elección del momento de las paradas en boxes, la selección del compuesto de neumáticos para cada relevo y la respuesta a eventos imprevistos, como la entrada del coche de seguridad o la aparición de condiciones meteorológicas cambiantes, pueden marcar diferencias de varias posiciones entre coches que disponen de un rendimiento puro prácticamente equivalente. Brawn y Parr (2016) describen, desde la experiencia directa de un director técnico de equipo campeón del mundo, cómo la inteligencia estratégica en tiempo real ha pasado a ser tan decisiva como la propia velocidad del monoplaza, especialmente desde la consolidación de los reglamentos técnicos modernos que han estrechado las diferencias mecánicas entre constructores.

La complejidad combinatoria del problema estratégico es elevada. Para una carrera de cincuenta vueltas con 20 vehículos en pista, el número de combinaciones plausibles entre el momento de las paradas, los compuestos disponibles, la respuesta a eventos aleatorios y las decisiones del resto de competidores supera con holgura cualquier capacidad de razonamiento puramente humano en el tiempo del que dispone un ingeniero de carrera durante el desarrollo de la prueba. Por este motivo, los equipos profesionales recurren desde finales de la década de los noventa a herramientas de simulación Monte Carlo (Metropolis y Ulam, 1949), que permiten enumerar miles de escenarios posibles y producir distribuciones de probabilidad sobre los resultados finales bajo cada estrategia candidata. Estas herramientas, integradas en la actividad cotidiana de boxes, son habitualmente desarrollos internos protegidos por secreto industrial y difícilmente accesibles para la comunidad investigadora o aficionada.

En el ámbito académico, el problema de la simulación de carreras de motorsport ha recibido una atención creciente en los últimos quince años. Bekker y Lotz (2009) establecieron una de las primeras formulaciones del problema mediante simulación de eventos discretos, y los

trabajos posteriores de Heilmeier et al. (2018, 2020) extendieron este enfoque incorporando degradación de neumáticos, consumo de combustible, maniobras de adelantamiento y, finalmente, la modelización probabilística de eventos como accidentes y fases de coche de seguridad mediante el método de Montecarlo. Otros autores han explorado la integración de redes neuronales para la toma de decisiones (Liu y Fotouhi, 2020), la optimización mediante programación dinámica y teoría de juegos (Aguad y Thraves, 2024) o, más recientemente, el aprendizaje por refuerzo explicable (Thomas et al., 2025). El conjunto de la literatura ha asentado un cuerpo metodológico sólido, pero presenta tres limitaciones recurrentes que dificultan su transferencia a un entorno de simulación interactiva como el que ofrecen los videojuegos de carreras. La primera es que prácticamente todos los trabajos publicados operan sobre datos históricos de la competición real obtenidos a través de interfaces como Ergast o FastF1, no sobre flujos de telemetría en tiempo real. La segunda es la granularidad: el modelo se construye habitualmente vuelta a vuelta, lo que impide capturar fenómenos que ocurren a escala de sector como las zonas de DRS, el efecto del aire sucio o los adelantamientos en zonas concretas del trazado. La tercera, quizá la más relevante para esta memoria, es que ninguno de los trabajos académicos revisados utiliza un videojuego comercial como plataforma experimental, dejando inexplorado un dominio que combina disponibilidad masiva de datos con experimentación controlada y reproducible.

Paralelamente, el ecosistema de la simulación de carreras como entretenimiento ha experimentado en la última década una expansión considerable. EA Sports F1 25 (Electronic Arts, 2025), el título oficial licenciado por la Federación Internacional del Automóvil dispone de una comunidad activa de varios millones de usuarios y emite de forma nativa por UDP toda la telemetría de la sesión a un puerto configurable. Esta característica, originalmente pensada para usuarios avanzados que conectan instrumentación física como volantes con retorno de fuerza, abre una vía de acceso a un volumen de datos comparable al de la categoría real, pero generados en un entorno totalmente controlado. A pesar de ello, las herramientas existentes en el ecosistema sim Racing, entre las que cabe destacar productos comerciales como F1Laps o Sim Racing Telemetry, se concentran en el análisis a posteriori del rendimiento del piloto y en la gestión de configuraciones del vehículo, sin abordar el problema de la simulación estratégica en tiempo real ni el diseño de una capa de interacción que devuelva al piloto las decisiones recomendadas durante la propia carrera.

A esta laguna se añade un factor que las publicaciones de hace apenas cinco años no podían contemplar: la disponibilidad madura de modelos de lenguaje de gran tamaño capaces de generar comunicaciones en lenguaje natural con latencia compatible con el desarrollo de una carrera. La interfaz tradicional entre un sistema de decisión estratégica y el piloto es el ingeniero de carrera humano, que reformula la salida cuantitativa del simulador en mensajes breves, contextualizados y emocionalmente calibrados que el piloto recibe por radio. Hasta la consolidación de los modelos de lenguaje generativos, esta función de traducción entre cifras y comunicación natural quedaba fuera del alcance de cualquier sistema automatizado. La posibilidad actual de delegar esta capa en un modelo generativo invita a replantear el problema completo, desde la captura de la telemetría hasta la entrega del mensaje al piloto, como un flujo cerrado de extremo a extremo.

El trabajo que aquí se presenta nace, por tanto, de la confluencia de tres factores: la relevancia incuestionable de la estrategia de carrera como problema técnico abierto, la disponibilidad de datos masivos en un entorno controlado a través del simulador F1 25 y la madurez reciente de los modelos de lenguaje como capa de interacción natural. La motivación última consiste en demostrar, mediante una prueba de concepto funcional, que es posible construir un ingeniero de carrera virtual completo, abierto y reproducible, sobre tecnologías accesibles a la comunidad académica y al entusiasta del simulador.

[placeholder]

Tabla 1. Usuarios potenciales de un documento de requisitos.

Perfil de audiencia		Necesidades
	Clientes usuarios y	Expresan sus necesidades y revisan el documento comprobando que lo incluido es correcto, ayudando a detectar posibles cambios.
	Gestores del proyecto	Emplean el documento para poder planificar el desarrollo y determinar los recursos y el tiempo necesarios para su construcción.
	Ingenieros del proyecto	Usan los requisitos para comprender las características del sistema que deben desarrollar.
	Ingenieros de pruebas	Emplean la especificación para definir las pruebas de validación.
	Ingenieros de mantenimiento	Utilizan el documento para comprender el sistema y las relaciones entre sus componentes.

Fuente: Adaptado de Sommerville (2011, p. 92).

1.2. Planteamiento del trabajo

La solución propuesta aborda el problema de la asistencia estratégica en carrera como un sistema distribuido de seis componentes funcionalmente especializados, conectados entre sí a través de una base de datos relacional con capacidades de mensajería transaccional. La premisa fundamental del diseño es que la estrategia óptima en un instante dado depende del estado completo de la carrera, posiciones, gaps, neumáticos, combustible, daños, condiciones meteorológicas, y de un modelo físico calibrado del comportamiento del simulador, que necesariamente se ajusta de forma empírica a partir de datos reales acumulados, dado que la implementación interna del juego es una caja negra propietaria.

El primer componente es un servicio de ingesta de telemetría escrito en Java sin dependencias de frameworks, que recibe los paquetes UDP emitidos por el simulador a una frecuencia configurable de hasta 60 Hz, los descodifica según la especificación oficial publicada por Electronic Arts (2025), y persiste en una base de datos Oracle AI Database 26ai una instantánea por cada sector completado. La elección de la granularidad por sector, en lugar de por vuelta completa o por paquete, responde a la voluntad de capturar fenómenos físicos que suceden a esa escala, activaciones de DRS, adelantamientos en zonas concretas del trazado, pérdida de tiempo por aire sucio en sectores con fuerte carga aerodinámica, sin incurrir en el coste de almacenar todos los paquetes en bruto, que a 20 Hz suponen del orden de cien filas por segundo y por vehículo.

El segundo componente es un módulo de calibración estadística desarrollado en Python que, a partir de los datos acumulados en sucesivas sesiones, ajusta los coeficientes del modelo de tiempo por sector. Este modelo expresa el tiempo de un sector como una función del rendimiento base del piloto, la edad y el compuesto del neumático, la carga de combustible, el estado de los daños aerodinámicos, la presencia de aire sucio y el efecto del DRS, entre otros factores. Una de las hipótesis fuertes que guían el diseño del módulo de calibración, sustentada en observaciones empíricas y en la información disponible en los foros técnicos de la comunidad, es que el simulador aplica modelos físicos diferenciados al coche del jugador y a los oponentes controlados por la inteligencia artificial del propio juego. Por este motivo, el

ajuste se realiza por separado para ambos regímenes y se almacenan dos conjuntos de coeficientes independientes.

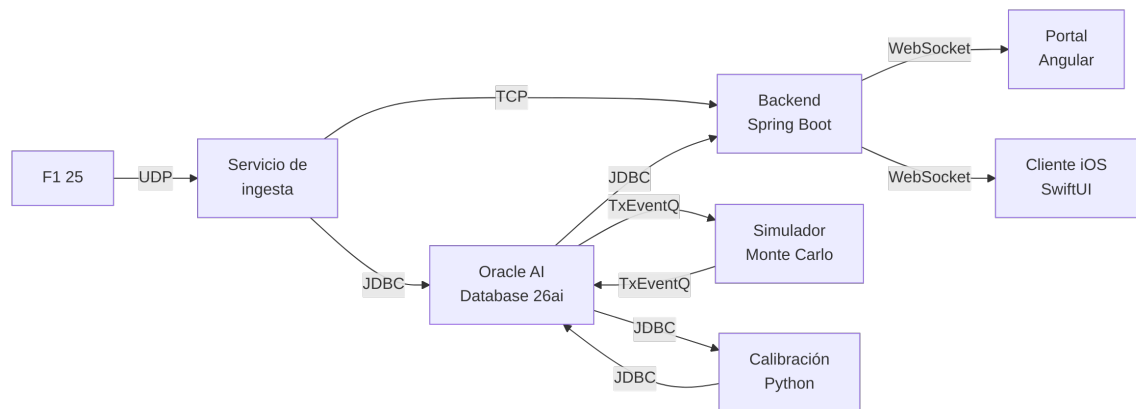
El tercer componente es el motor de simulación Monte Carlo, desarrollado en Python sobre FastAPI, que dado un estado de carrera y un conjunto de estrategias candidatas produce, para cada una de ellas, una distribución de probabilidad sobre las posiciones finales de los 20 vehículos. Cada iteración Monte Carlo simula la totalidad de los sectores restantes vehículo a vehículo, resolviendo en cada paso las interacciones entre coches adyacentes y propagando los efectos de los eventos aleatorios. La agregación de varios miles de iteraciones permite construir distribuciones suficientemente estables como para fundamentar una decisión estratégica.

El cuarto componente es una capa de coordinación basada en colas transaccionales nativas de Oracle AI Database 26ai (Oracle Corporation, 2025b), conocidas como TxEventQ. Esta elección desacopla los componentes entre sí: ni el módulo de calibración ni el motor de simulación necesitan conocer la existencia del servicio de ingesta o del backend, y todos los disparadores entre componentes viajan como mensajes encolados con garantía de entrega exactamente una vez. Esta decisión tiene una implicación arquitectónica relevante que se discute en detalle en el capítulo dedicado a la arquitectura propuesta.

El quinto componente es un portal web desarrollado en Angular que permite al usuario seguir en directo el desarrollo de la carrera, consultar el estado de la calibración, lanzar simulaciones manualmente y comparar los resultados de distintas estrategias candidatas. Y el sexto componente es un cliente nativo iOS desarrollado en SwiftUI, que recibe los mensajes del ingeniero de carrera virtual a través de una conexión por websocket y los reproduce mediante síntesis de voz en el dispositivo móvil del piloto, emulando así la radio que conecta al equipo con el coche en una carrera real.

A continuación, la disposición general de los seis componentes y los flujos principales de información entre ellos.

Figura 1. Visión general del sistema y flujos principales entre componentes.



Fuente: Elaboración propia.

El alcance del trabajo se enmarca explícitamente como una prueba de concepto: el sistema es funcional y demuestra la viabilidad técnica del enfoque, sin aspirar a la precisión que un equipo profesional obtendría con sus propias herramientas internas. Esta elección responde a la naturaleza académica del proyecto y a la necesidad de acotar el alcance experimental dentro del horizonte temporal disponible.

1.3. Estructura del trabajo

A partir de este capítulo, el resto de la memoria se organiza en seis capítulos adicionales y los apartados finales de referencias y anexos. La distribución del contenido sigue el orden lógico que va desde la fundamentación teórica del problema hasta la evaluación de los resultados obtenidos.

En el Capítulo 2 se presenta el contexto y el estado del arte. Se recorre la evolución histórica de la estrategia de carrera en Fórmula 1, desde las primeras herramientas estadísticas utilizadas por los equipos en los años setenta hasta los sistemas actuales basados en simulación Monte Carlo en tiempo real. A continuación, se revisa la literatura académica sobre simulación de motorsport, análisis de telemetría de sim Racing como entorno experimental y arquitecturas de agentes basados en modelos de lenguaje, identificando para cada línea las contribuciones más relevantes y las lagunas que justifican la novedad del trabajo presentado.

El Capítulo 3 formaliza los objetivos del trabajo y describe la metodología seguida para alcanzarlos. Se distingue entre un objetivo general, formulado en términos del sistema completo, y seis objetivos específicos numerados que descomponen el trabajo en contribuciones técnicamente acotadas. Cada objetivo específico es trazable a un capítulo posterior de la memoria y a la evaluación experimental que se presenta en el Capítulo 6.

En el Capítulo 4 se detalla la arquitectura propuesta del sistema. Se justifican las decisiones de diseño más relevantes, incluyendo la elección de Java sin frameworks para el componente de ingesta, el uso de JDBC sobre Oracle AI Database 26ai como capa de persistencia, la adopción de TxEventQ como mecanismo de coordinación entre componentes y la granularidad por sector como unidad temporal del modelo. Cada decisión se discute frente a alternativas razonables y se argumenta en términos de los requisitos no funcionales del sistema.

El Capítulo 5 describe la implementación concreta de los seis componentes. Se documentan los algoritmos clave del módulo de calibración estadística, la mecánica del motor de simulación Monte Carlo, el esquema de la base de datos, los protocolos de comunicación entre componentes y el catálogo de mensajes de voz generados por el modelo de lenguaje. El énfasis se sitúa en las decisiones técnicas que tienen impacto medible en el comportamiento global del sistema.

En el Capítulo 6 se presentan los resultados de la evaluación experimental. La validación se ha articulado en cuatro ejes: la latencia extremo a extremo del pipeline desde la captura del paquete UDP hasta la reproducción del mensaje de voz en el dispositivo del piloto, la convergencia del motor Monte Carlo a medida que se incrementa el número de iteraciones, el ajuste estadístico del modelo de calibración medido como coeficiente de determinación separado para los regímenes del jugador y de la inteligencia artificial del juego, y la valoración cualitativa de la plausibilidad de los mensajes de voz generados.

Finalmente, el Capítulo 7 recoge las conclusiones del trabajo, evaluando el grado de cumplimiento de cada uno de los objetivos específicos planteados en el Capítulo 3, e identifica las líneas de trabajo futuro que el proyecto deja abiertas. Estas líneas incluyen tanto cuestiones de investigación pendientes en el modelo de calibración como posibilidades de evolución del sistema hacia escenarios de uso más amplios que el contemplado en esta prueba de concepto.

2. Contexto y Estado del Arte

[placeholder]

2.1. Estrategia de carrera en Fórmula 1

La estrategia ha pasado de ser un complemento del rendimiento mecánico a convertirse en un factor de victoria por derecho propio. Brawn y Parr (2016) describen cómo, desde finales de los noventa, los equipos punteros incorporaron en boxes herramientas de simulación Monte Carlo (Metropolis y Ulam, 1949) que permitían enumerar miles de escenarios y caracterizar la robustez de cada decisión frente a eventos aleatorios como la entrada del coche de seguridad o un fallo mecánico. El segundo gran salto llegó en 2001, cuando la FIA empezó a distribuir cronometraje electrónico en directo a todos los equipos: a partir de ese momento, las simulaciones podían ejecutarse durante la propia carrera, alimentadas por el estado actualizado de cada coche.

La descripción contemporánea del problema puede consultarse en Rubinstein y Kroese (2017), referencia estándar sobre simulación estocástica. Las herramientas concretas que utilizan los equipos profesionales son, sin excepción, desarrollos internos protegidos por secreto industrial. Lo que se conoce públicamente procede de declaraciones puntuales de directores técnicos y de los pocos trabajos académicos abiertos, situación que solo ha empezado a corregirse en la última década.

2.2. Simulación académica de carreras

Bekker y Lotz (2009) propusieron la primera formulación académica sistemática del problema mediante simulación de eventos discretos sobre tres carreras de la temporada 2005. Establecieron el patrón conceptual que la literatura posterior ha conservado: la carrera se discretiza en unidades temporales, vueltas completas, y en cada unidad se actualiza el estado de cada vehículo en función de las decisiones estratégicas y de los eventos imprevistos.

La aportación más completa al problema con técnicas de Monte Carlo se debe al grupo de Heilmeier en la Universidad Técnica de Múnich. Heilmeier et al. (2018, 2020) integraron en sus simuladores la degradación de neumáticos, el consumo de combustible, los adelantamientos en pista y, finalmente, distribuciones de probabilidad calibradas sobre los

principales fenómenos aleatorios de la categoría: accidentes, fases de coche de seguridad, dispersión de los tiempos en parada y variabilidad del rendimiento del piloto en la salida. Su base de datos abierta sobre la era híbrida 2014-2019 sigue siendo una referencia obligada.

A partir de 2020 la literatura ha incorporado técnicas de aprendizaje automático: Liu y Fotouhi (2020) aplicaron redes neuronales y búsqueda de árbol Monte Carlo a Fórmula E, Aguad y Thraves (2024) abordaron el problema mediante programación dinámica y teoría de juegos, y Thomas et al. (2025) propusieron recientemente un sistema de aprendizaje por refuerzo explicable específico para Fórmula 1. Todos estos trabajos comparten una característica con sus predecesores: operan a granularidad de vuelta completa sobre datos históricos de la competición real, lo que deja sin capturar fenómenos relevantes que ocurren a escala de sector, activaciones de DRS, adelantamientos en zonas concretas, pérdidas por aire sucio en sectores específicos. Esta limitación es precisamente la que este trabajo aborda mediante una granularidad más fina.

2.3. Sim Racing como entorno de investigación

El simulador comercial ha emergido en los últimos años como plataforma controlada para el estudio del comportamiento del piloto, con la ventaja de ofrecer condiciones reproducibles y datos masivos sin la complejidad logística del motorsport real. Hojaji et al. (2024) presentaron el estudio más extenso de la línea: ciento setenta y cuatro participantes y mil trescientas vueltas en Brands Hatch sobre Assetto Corsa Competizione, con el objetivo de predecir el nivel de rendimiento del piloto a partir de variables de telemetría. El trabajo demuestra empíricamente que los datos de un simulador comercial son lo suficientemente ricos como para soportar análisis estadísticos no triviales.

Dentro del ecosistema sim Racing, EA Sports F1 25 (Electronic Arts, 2025) ocupa una posición particular. Es el juego oficialmente licenciado por la FIA, dispone de una comunidad activa de varios millones de usuarios y emite de forma nativa por UDP toda la telemetría de la sesión a un puerto configurable, con dieciséis tipos de paquete a frecuencias de hasta 60 Hz. Esta característica abre la posibilidad de utilizar el simulador como fuente de datos masivos de calidad equivalente a la que un equipo profesional obtiene de su propio coche.

Las herramientas existentes en el ecosistema, F1Laps, Sim Racing Telemetry, ofrecen funcionalidades sofisticadas para el análisis a posteriori del rendimiento del piloto, pero ninguna aborda el problema de la simulación estratégica en tiempo real ni propone una capa de interacción que devuelva al piloto recomendaciones durante la propia carrera. Esa combinación de disponibilidad masiva de datos y ausencia de herramientas estratégicas es el hueco que este trabajo se propone cubrir.

2.4. Modelos de lenguaje como capa de interacción

La elaboración de este trabajo coincide con un momento de madurez técnica de los modelos de lenguaje de gran tamaño que está modificando la forma de concebir las interfaces hombre-máquina en sistemas de soporte a la decisión. Las revisiones recientes (Lin et al., 2025) coinciden en identificar una evolución desde el uso de los modelos como simples generadores de texto hacia su incorporación como núcleo de razonamiento de sistemas autónomos capaces de planificar tareas, invocar herramientas externas y mantener estado entre interacciones.

La aplicación de LLMs a entornos donde las decisiones tienen consecuencias significativas plantea compromisos entre flexibilidad y fiabilidad. Carrero et al. (2026), en una evaluación de sistemas de agentes para soporte clínico, encontraron mejoras moderadas frente al modelo base, pero advirtieron del coste computacional y de los problemas de latencia de las arquitecturas multiagente más complejas. La línea común a estos trabajos es la conveniencia de delegar la lógica crítica a componentes deterministas y reservar el modelo de lenguaje para las funciones donde su flexibilidad expresiva aporta valor sin comprometer la fiabilidad del sistema.

Este trabajo se inscribe en esta línea conservadora: el motor cuantitativo permanece confiado a componentes deterministas y el modelo de lenguaje se reserva para la capa de comunicación, donde transforma resultados estructurados en mensajes naturales a partir de un catálogo cerrado de plantillas. La latencia típica de los modelos accesibles por interfaz remota, del orden de cientos de milisegundos por mensaje, es perfectamente compatible con el ritmo temporal de los eventos estratégicos relevantes en una carrera.

2.5. Síntesis y posicionamiento

La revisión anterior permite identificar dos lagunas concretas que motivan este trabajo. La literatura académica sobre simulación de carreras ha consolidado una metodología sólida, pero opera casi siempre sobre datos históricos de la competición real y a granularidad de vuelta completa, sin contemplar la posibilidad de utilizar un simulador comercial como fuente de datos en tiempo real ni de modelar el problema a escala de sector. Por su parte, las herramientas existentes en el ecosistema del simulador F1 25 ofrecen un análisis a posteriori sofisticado, pero ninguna aborda la simulación estratégica en tiempo real ni propone una capa de interacción que devuelva al piloto las recomendaciones durante la propia carrera.

A estas dos lagunas se suma una posibilidad técnica que apenas dos años atrás no existía: la madurez reciente de los modelos de lenguaje habilita una capa de comunicación natural entre el sistema de decisión y el piloto, similar a la radio que conecta a un equipo real con su coche, dentro de una arquitectura híbrida donde la lógica cuantitativa sigue confiada a componentes deterministas. La combinación de simulación estocástica, granularidad de sector, calibración empírica sobre un simulador comercial y entrega por voz mediante LLM no aparece, hasta donde la revisión bibliográfica permite afirmar, en ningún trabajo previo.

3. Objetivos y metodología de trabajo

[placeholder]

3.1. Objetivo general

El objetivo general se enuncia como sigue: diseñar, implementar y validar empíricamente un sistema completo de asistencia estratégica para pilotos del simulador EA Sports F1 25, capaz de operar en tiempo real durante el desarrollo de una carrera, que integre la captura continua de telemetría, la calibración estadística empírica del modelo físico subyacente, la simulación Monte Carlo de estrategias candidatas y la entrega de recomendaciones al piloto mediante mensajes de voz generados por un modelo de lenguaje.

Este objetivo delimita cuatro dimensiones del alcance: el sistema debe cubrir la cadena completa entre simulador y piloto sin dejar eslabones fuera; debe operar en tiempo real con latencias compatibles con los eventos estratégicos relevantes; debe calibrarse empíricamente sobre datos del propio simulador, dado que su implementación interna es propietaria; y debe entregar las recomendaciones por voz para emular la función comunicativa de un ingeniero de carrera real.

3.2. Objetivos específicos

Los seis objetivos específicos siguen el principio SMART (Doran, 1981): cada uno es concreto, medible, alcanzable, relevante y acotado en el tiempo. Cada objetivo admite valoración independiente, y su consecución conjunta basta para considerar logrado el objetivo general.

OE1. Diseñar e implementar un servicio de ingesta capaz de procesar, en tiempo real y sin pérdida de datos en condiciones nominales, la telemetría UDP emitida por el simulador EA Sports F1 25 a una frecuencia de hasta 20 Hz, con persistencia transaccional sobre Oracle AI Database 26ai y captura selectiva de instantáneas en los sectores completados de cada vehículo de la parrilla.

OE2. Modelar empíricamente el comportamiento físico del simulador en lo relativo a los principales factores que afectan al tiempo por sector, degradación y temperatura del neumático, consumo y peso del combustible, daño aerodinámico, efecto del aire sucio, ventaja del DRS y condiciones meteorológicas, mediante un módulo de calibración estadística que ajuste los coeficientes correspondientes a partir de los datos acumulados en sesiones

reales del simulador, distinguiendo de forma sistemática los regímenes físicos correspondientes al coche del jugador humano y a los oponentes controlados por la inteligencia artificial del juego.

OE3. Implementar un motor de simulación Monte Carlo que, dado un estado de carrera y un conjunto de estrategias candidatas de paradas en boxes, produzca distribuciones de probabilidad sobre las posiciones finales de los 20 vehículos de la parrilla, con un número de iteraciones suficiente para garantizar la convergencia estadística de las distribuciones predichas y a una granularidad por sector que permita capturar los efectos físicos a esta escala temporal.

OE4. Diseñar una arquitectura desacoplada entre los componentes de ingesta, persistencia, calibración y simulación mediante el uso de colas transaccionales nativas de Oracle AI Database 26ai, de modo que cada componente pueda evolucionar, desplegarse y operar independientemente del resto, y que los flujos críticos del sistema puedan ofrecer garantías de entrega exactamente una vez sin intervención adicional de la capa de aplicación.

OE5. Construir una capa de salida que traduzca los resultados cuantitativos de la simulación en mensajes de voz contextualmente pertinentes y emocionalmente calibrados al estilo de un ingeniero de carrera profesional, generados mediante un modelo de lenguaje y entregados al piloto a través de un cliente nativo iOS que reproduzca las indicaciones por síntesis de voz durante el desarrollo de la propia carrera.

OE6. Validar el sistema completo mediante una evaluación experimental que contemple, como mínimo, la latencia extremo a extremo del pipeline desde la captura del paquete UDP hasta la reproducción del mensaje en el dispositivo del piloto, la convergencia estadística del motor Monte Carlo en función del número de iteraciones, el ajuste estadístico del modelo de calibración medido como coeficiente de determinación separado para los regímenes del jugador y de la inteligencia artificial del juego, y la valoración cualitativa de la plausibilidad y la pertinencia contextual de los mensajes de voz generados.

Los objetivos OE1 a OE5 se corresponden con los capítulos 4 y 5 (arquitectura e implementación). El OE6 articula el Capítulo 6 (evaluación). Las conclusiones del Capítulo 7 retomarán cada objetivo por separado.

3.3. Metodología de trabajo

La metodología seguida para el desarrollo del trabajo se inscribe en el marco de los ciclos de desarrollo iterativo e incremental ampliamente utilizados en la ingeniería de software contemporánea. La elección de esta aproximación responde a dos consideraciones de fondo. La primera es la naturaleza fuertemente experimental del proyecto: el modelo físico del simulador es desconocido y debe descubrirse empíricamente a partir de los datos, lo que hace inviable cualquier metodología de tipo cascada que exija fijar una especificación detallada antes de comenzar la implementación. La segunda es la interdependencia entre componentes: las decisiones de diseño tomadas en el módulo de calibración condicionan la información que el módulo de simulación necesita recibir de la base de datos, y esta a su vez condiciona el formato de las instantáneas que el servicio de ingesta debe producir. Solo un ciclo iterativo en el que los componentes evolucionan en paralelo y se ajustan mutuamente permite gestionar este tipo de dependencias.

El trabajo se ha organizado en cinco fases consecutivas, cada una de las cuales ha producido entregables tangibles y ha servido de base para la siguiente. La descripción que sigue resume las fases en términos de las actividades principales, los entregables producidos y las herramientas utilizadas.

3.3.1. Fase de análisis del problema y revisión bibliográfica

La primera fase del trabajo se dedicó a la comprensión profunda del problema y al recorrido de la literatura existente. Las actividades principales consistieron en el estudio de la especificación técnica del protocolo UDP del simulador F1 25, el repaso de las publicaciones académicas sobre simulación de carreras de motorsport, la familiarización con las herramientas existentes en el ecosistema sim racing y la identificación de las lagunas concretas que el trabajo se proponía abordar. El entregable principal de esta fase fue el contenido que se ha presentado en los capítulos primero y segundo de esta memoria, junto con un primer esbozo de los objetivos específicos del proyecto.

3.3.2. Fase de diseño arquitectónico

Sobre la base del análisis previo se procedió al diseño de la arquitectura general del sistema. Las decisiones tomadas en esta fase incluyeron la elección del lenguaje de programación más adecuado para cada componente, la definición de las interfaces entre componentes, la

elección del sistema de persistencia y de la estrategia de coordinación entre módulos. Esta fase produjo como entregables los diagramas de arquitectura, los esquemas de la base de datos y los registros de decisiones arquitectónicas que justifican cada elección frente a las alternativas consideradas. El contenido de esta fase se desarrolla en el Capítulo 4.

3.3.3. Fase de implementación incremental

La implementación del sistema se llevó a cabo de forma incremental siguiendo el orden natural del flujo de datos: en primer lugar el servicio de ingesta de telemetría, a continuación el esquema de la base de datos y los procedimientos de persistencia, después el módulo de calibración estadística, posteriormente el motor de simulación Monte Carlo, a continuación la capa de coordinación entre componentes mediante colas transaccionales y, finalmente, las interfaces de salida hacia el portal web y el cliente iOS. Cada componente se implementó hasta alcanzar un nivel funcional mínimo viable, se integró con los componentes ya existentes y se sometió a pruebas básicas de funcionamiento antes de pasar al siguiente. El contenido de esta fase se documenta en el Capítulo 5.

3.3.4. Fase de calibración y ajuste empírico

Una vez completada la implementación del sistema en su nivel funcional mínimo viable, se procedió a la captura sistemática de datos a partir de varias sesiones de carrera completas en el simulador F1 25, ejecutadas en distintos circuitos y bajo distintas condiciones para garantizar la diversidad del conjunto de datos resultante. Estos datos sirvieron para alimentar el módulo de calibración y para ajustar empíricamente los coeficientes del modelo físico, separando los datos correspondientes al coche del jugador de los correspondientes a los oponentes controlados por la inteligencia artificial del juego. Las decisiones metodológicas tomadas en esta fase se discuten con detalle en el Capítulo 5, en la sección dedicada al módulo de calibración.

3.3.5. Fase de evaluación experimental

La fase final del trabajo consistió en la evaluación experimental del sistema completo según los cuatro ejes definidos en el objetivo OE6. Para cada uno de los ejes se diseñó un protocolo de medición específico, se ejecutaron las pruebas correspondientes y se analizaron los resultados obtenidos. Los resultados se presentan en el Capítulo 6 acompañados de las gráficas y tablas necesarias para su interpretación.

3.3.6. Herramientas y entornos de desarrollo

El servicio de ingesta se ha desarrollado en Java 23 (Oracle Corporation, 2024) sin frameworks, dado que la tarea, escucha de un socket UDP y persistencia mediante JDBC, no requiere inyección de dependencias ni configuración automática. Los módulos de calibración y simulación se han implementado en Python 3.12 con FastAPI (Ramírez, 2025) como servidor del motor de simulación. La capa de persistencia y coordinación se asienta sobre Oracle AI Database 26ai (Oracle Corporation, 2025), elegida por su soporte transaccional, la disponibilidad nativa de colas TxEventQ y las capacidades específicas para inteligencia artificial que la versión 26ai consolida. El portal web utiliza Angular 21 (Google, 2025) y el cliente iOS está desarrollado en SwiftUI con AVSpeechSynthesizer (Apple Inc., s.f.) como motor de síntesis de voz. El código se gestiona con Git y se publica en GitHub bajo licencia MIT.

4. Título del capítulo

[placeholder]

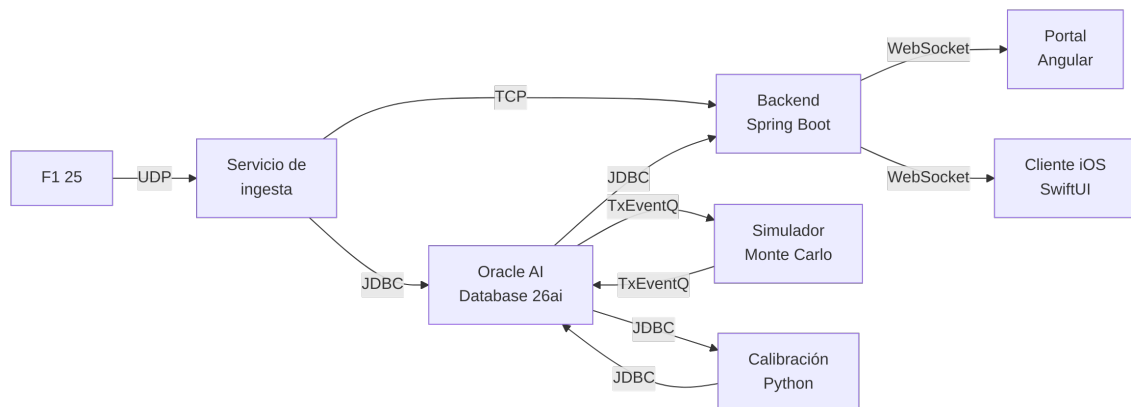
4.1. Visión general del sistema

El sistema se articula en seis componentes con responsabilidades especializadas e interfaces explícitas. La premisa de diseño es la separación estricta entre flujos críticos en tiempo, ingesta de telemetría y entrega de mensajes al piloto, y flujos analíticos diferidos, calibración estadística y simulación Monte Carlo, de modo que las exigencias de rendimiento de los primeros no impongan restricciones sobre la complejidad de los segundos. Los seis componentes son los siguientes:

- Servicio de ingesta (Java 23, JDBC). Recibe UDP del simulador y persiste instantáneas por sector en la base de datos.
- Base de datos (Oracle AI Database 26ai). Almacén persistente y capa de coordinación entre componentes mediante TxEventQ.
- Backend (Spring Boot 3.5). Expone API REST y WebSocket; orquesta los servicios diferidos de calibración y simulación.
- Servicio de calibración (Python 3.12). Ajusta estadísticamente los coeficientes del modelo físico a partir de los datos acumulados.
- Simulador Monte Carlo (Python, FastAPI). Motor de simulación estocástica y generador de estrategias candidatas.
- Interfaces de salida (Angular 21, SwiftUI). Portal web de seguimiento y cliente iOS para entrega de mensajes por voz.

La Figura se reproduce aquí para facilitar la lectura.

Figura 2. Visión general del sistema y flujos principales entre componentes.



Fuente: Elaboración propia.

4.2. Decisiones arquitectónicas fundamentales

Esta sección recoge las seis decisiones arquitectónicas con mayor impacto sobre el comportamiento global del sistema. Cada una se presenta con una descripción breve del problema, la opción adoptada y las alternativas descartadas.

4.2.1. Java sin frameworks para el componente de ingesta

El servicio de ingesta tiene una responsabilidad estrecha: escuchar un socket UDP, parsear los bytes según la especificación binaria y escribir filas en la base de datos. Para esta tarea se ha implementado en Java 23 puro, sin Spring Boot. La razón principal es que el componente no necesita ninguna de las facilidades habituales de un framework de aplicación: ni inyección de dependencias, ni configuración automática, ni soporte HTTP. La ausencia de framework se traduce en un arranque de fracciones de segundo frente a los varios segundos típicos de una aplicación Spring Boot equivalente, en una reducción drástica del número de dependencias (con la consiguiente simplificación del ciclo de vida y reducción de la superficie de ataque) y en una lógica que queda completamente a la vista del lector: un bucle de escucha, un dispatcher por tipo de paquete y un escritor JDBC, sin capas intermedias. En el backend, donde las funciones REST y WebSocket son centrales, sí se ha optado por Spring Boot.

4.2.2. Acceso a base de datos mediante JDBC frente a ORMs

Tanto el servicio de ingesta como el backend utilizan JDBC con sentencias preparadas en lugar de un ORM como JPA o Hibernate. La razón principal es la naturaleza de la carga: la ingesta es

esencialmente inserción intensiva (varias decenas de filas por vuelta y vehículo) y el coste por fila es crítico. JDBC con prepared statements y batching ofrece el camino más directo, sin la sobrecarga de cache de primer nivel, dirty checking y traducción objeto-relacional que introduce un ORM.

A esto se añade que las tablas son intencionadamente planas: sector_snapshots tiene unas cuarenta columnas, pero ninguna relación uno-a-muchos hacia tablas hijas. El valor añadido de un ORM, gestión de relaciones, carga perezosa, cascadas, no aplica aquí, y los costes en líneas de código de configuración y entidades superan holgadamente los beneficios.

Para mitigar el riesgo de errores con índices numéricos en las llamadas a las sentencias preparadas, se ha adoptado un patrón basado en records de Java 23 (Oracle Corporation, 2024): cada tabla se representa por un record inmutable que encapsula los valores de una fila y expone un método de inserción en el que la SQL y las llamadas a setXxx() quedan localizadas en un único lugar. Ofrece seguridad de tipos comparable a la de un ORM sin introducir dependencias externas.

4.2.3. Granularidad por sector como unidad temporal

A diferencia de la literatura académica revisada, que opera por vuelta completa, este sistema modela a granularidad de sector. La justificación es doble: el simulador expone los tiempos por sector como variable nativa del protocolo, sin coste adicional de procesamiento; y varios fenómenos físicos clave operan a escala de sector y no de vuelta, las zonas de DRS están definidas sobre sectores específicos, el aire sucio se manifiesta de forma muy desigual entre sectores aerodinámicos y los adelantamientos ocurren en zonas concretas del trazado.

El coste es evidente: cada iteración Monte Carlo ejecuta tres veces más pasos por vuelta y la resolución de interacciones entre coches adyacentes se propaga sector a sector. Este coste se ha asumido como parte del compromiso, y su impacto sobre el tiempo de simulación se discute en el Capítulo 6.

4.2.4. Calibración dual: regímenes físicos del jugador y de la IA del juego

El sistema mantiene dos conjuntos separados de coeficientes calibrados: uno para el coche del jugador y otro para los oponentes controlados por la IA del simulador. La hipótesis subyacente, observada informalmente y respaldada por la información disponible en los foros técnicos de la comunidad, es que el juego aplica a los oponentes un modelo físico simplificado,

con degradación de neumáticos y consumo de combustible que siguen curvas predeterminadas, mientras que el coche del jugador está sujeto a un modelo más completo que responde a las acciones del piloto humano. Ajustar un único modelo sobre datos mezclados produciría coeficientes intermedios que no representarían correctamente ninguno de los dos regímenes.

La validación empírica de esta hipótesis se presenta en el Capítulo 6: si el R^2 del modelo del jugador resulta significativamente inferior al de la IA, la decisión queda justificada empíricamente.

4.2.5. Coordinación entre componentes mediante colas transaccionales

El sistema utiliza colas transaccionales nativas de Oracle AI Database 26ai (TxEventQ) como mecanismo de coordinación entre componentes, en lugar de un broker externo (Kafka, RabbitMQ) o llamadas REST directas. La elección se fundamenta en cuatro razones: TxEventQ está integrado en la propia base, eliminando la necesidad de desplegar y operar un componente adicional; las operaciones de encolado participan en las transacciones de la base de datos, garantizando atomicidad entre escritura y mensaje sin patrones outbox; las garantías de entrega exactamente una vez son nativas; y la versión 26ai consolida una interfaz compatible con Kafka (Oracle Corporation, 2025b), lo que deja abierta la posibilidad de migrar a un broker externo en el futuro manteniendo el modelo de programación.

Las llamadas REST directas se descartaron por la pérdida de garantías de entrega ante caídas momentáneas del consumidor y por el acoplamiento temporal entre productor y consumidor, que limita la posibilidad de evolucionar y desplegar componentes de forma independiente.

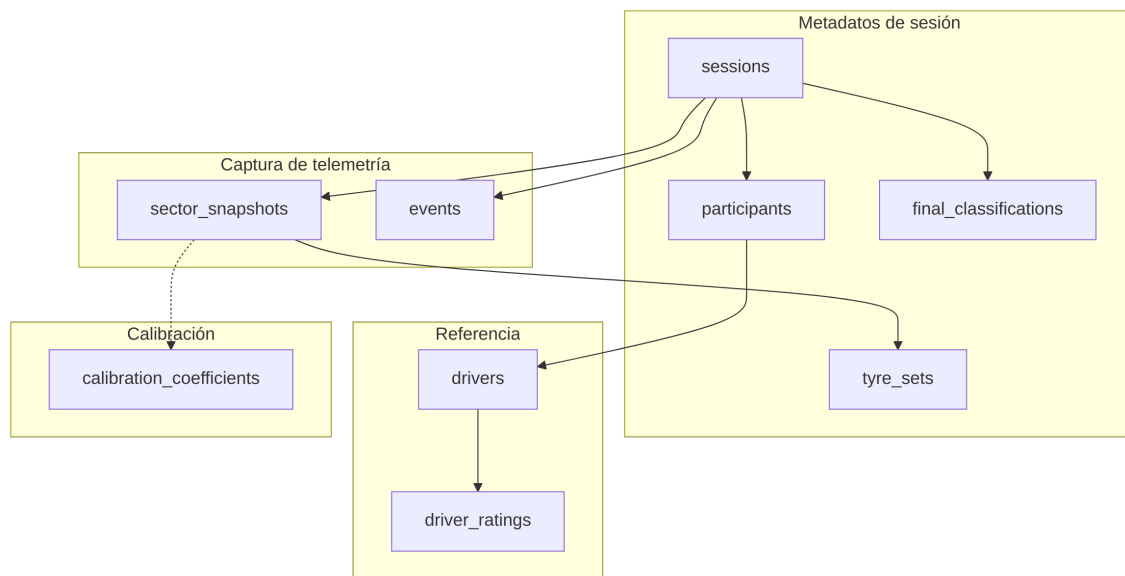
4.2.6. Desnormalización selectiva del esquema relacional

Varias columnas de la tabla central `sector_snapshots` están deliberadamente desnormalizadas: las condiciones meteorológicas, el estado del neumático y los daños del coche se almacenan como columnas planas en lugar de en tablas separadas. Los datos son específicos del momento de captura y no se reutilizan en otras filas, por lo que la normalización no ahorraría almacenamiento. Y el patrón de acceso dominante sobre esta tabla son consultas analíticas que necesitan todas estas columnas a la vez para alimentar la calibración o la simulación: la desnormalización elimina operaciones de JOIN que ralentizarían estas consultas sobre conjuntos de varios millones de filas.

4.3. Modelado de datos

El modelo de datos del sistema se compone de nueve tablas principales que cubren todos los aspectos del problema desde el dato bruto capturado del simulador hasta los coeficientes calibrados que el motor de simulación necesita. La Figura 2 representa esquemáticamente las nueve tablas y las relaciones entre ellas.

Figura 3. Modelo de datos del sistema. Las tablas se agrupan según su papel funcional: captura de telemetría, calibración y metadatos.



Fuente: Elaboración propia.

La tabla central del sistema es `sector_snapshots`, que almacena una fila por cada sector completado de cada vehículo, en cada vuelta de cada sesión. Su clave primaria es la combinación natural de cuatro columnas, identificador único de la sesión, índice del coche, número de vuelta y número de sector, que garantiza la unicidad sin necesidad de introducir una clave subrogada. La elección de una clave primaria natural en lugar de una clave subrogada de tipo identidad responde tanto al ahorro de una columna por fila como a las facilidades que ofrece para detectar y descartar inserciones duplicadas en escenarios de recuperación tras pérdida de paquetes UDP, dado que el comportamiento por defecto del sistema gestor de la base de datos ante la violación de la clave primaria, el rechazo silencioso de la inserción duplicada, es exactamente lo que el flujo de recuperación necesita.

Las tablas `sessions`, `participants`, `events` y `final_classifications` complementan a `sector_snapshots` con información que cambia con menor frecuencia: los metadatos de cada sesión (circuitos, condiciones, configuración del juego), los pilotos y equipos participantes, los eventos discretos que ocurren durante la sesión (entradas del coche de seguridad, banderas, abandonos) y las clasificaciones finales que el simulador emite al término de la carrera. Las tablas `drivers` y `driver_ratings` contienen información de calidad de pilotos que se utiliza para ponderar las observaciones durante el ajuste estadístico. La tabla `tyre_sets` mantiene el inventario de juegos de neumáticos disponibles para cada vehículo y su historial de uso. Y la tabla `calibration_coefficients` almacena los coeficientes ajustados por el módulo de calibración, organizados por circuito, régimen físico (jugador o inteligencia artificial), versión del ajuste y fecha de generación.

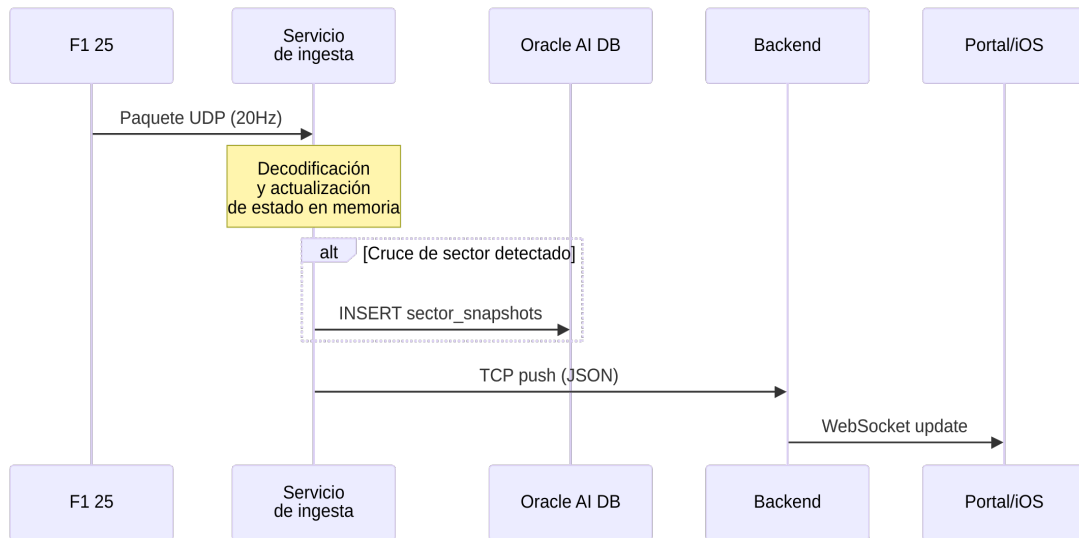
4.4. Flujos de información

El sistema articula sus operaciones en torno a cuatro flujos principales: ingesta y persistencia, calibración, simulación y salida hacia los clientes. Las figuras 3, 4 y 5 ilustran respectivamente los tres primeros flujos como diagramas de secuencia. La descripción detallada de cada componente se difiere al Capítulo 5.

4.4.1. Flujo de ingesta y persistencia

El servicio de ingesta recibe los paquetes UDP del simulador, los descodifica, actualiza el estado en memoria de cada vehículo y persiste una nueva instantánea cada vez que un coche completa un sector. En paralelo, empuja al backend por TCP el estado en directo que el portal y el cliente iOS necesitan para representar el desarrollo de la carrera (Figura 3).

Figura 4. Flujo de ingesta y persistencia. El servicio de ingesta procesa los paquetes UDP del simulador, persiste una instantánea por sector completado y notifica el estado en directo al backend.

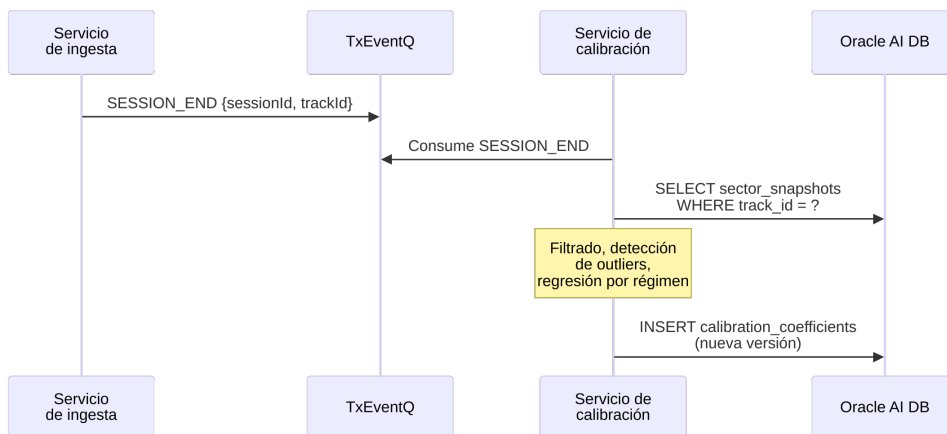


Fuente: Elaboración propia.

4.4.2. Flujo de calibración

La calibración se dispara al término de cada sesión, automáticamente desde el servicio de ingesta o manualmente desde el portal. El servicio de calibración consume el mensaje TxEventQ correspondiente, lee las instantáneas del circuito en cuestión, ejecuta el procedimiento de ajuste estadístico y persiste los nuevos coeficientes como una nueva versión, sin sobrescribir las anteriores (Figura 4).

Figura 5. Flujo de calibración disparado al término de una sesión. El servicio consume el mensaje encolado, lee las instantáneas del circuito, ejecuta el ajuste estadístico y persiste una nueva versión de los coeficientes.

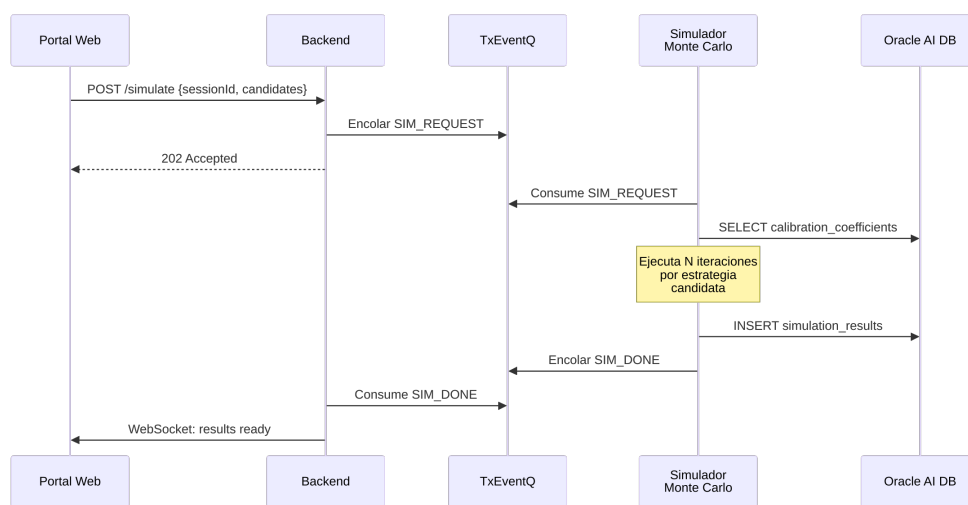


Fuente: Elaboración propia.

4.4.3. Flujo de simulación

La simulación se activa cuando el usuario solicita la evaluación de un conjunto de estrategias candidatas. La petición se traduce en un mensaje TxEventQ que el motor consume, ejecuta el número configurado de iteraciones para cada candidata y devuelve los resultados al backend, que los notifica al cliente por WebSocket (Figura 6).

Figura 6. Flujo de simulación Monte Carlo desencadenado desde el portal web. La petición se desacopla del cliente mediante TxEventQ y los resultados se devuelven asincrónicamente por WebSocket.



Fuente: Elaboración propia.

4.4.4. Flujo de salida hacia portal y cliente iOS

El backend conecta con las interfaces de usuario por dos mecanismos complementarios. El portal Angular utiliza llamadas REST para los datos históricos y una conexión WebSocket para las actualizaciones en directo. El cliente iOS utiliza exclusivamente WebSocket sobre el que recibe los mensajes generados por el ingeniero virtual, los pasa al motor de síntesis de voz del sistema operativo y los reproduce al piloto.

4.5. Consideraciones sobre seguridad y gestión del estado

El alcance del sistema como prueba de concepto en un entorno doméstico permite simplificar varias dimensiones que en un despliegue de producción exigirían un tratamiento más cuidadoso. La autenticación entre componentes se asume confiada al hecho de que todos ellos ejecutan en la red local del usuario; las comunicaciones entre el servicio de ingesta y el

backend, entre el backend y los clientes y entre los servicios y la base de datos no incorporan cifrado en transporte. La gestión del estado de las sesiones se basa en el identificador único que el propio simulador genera para cada partida y que el servicio de ingesta utiliza como agrupador de todas las filas correspondientes a esa sesión, lo que elimina la necesidad de un mecanismo explícito de sesión por parte del sistema. Estas simplificaciones se documentan explícitamente en el capítulo dedicado al trabajo futuro como elementos a reconsiderar en cualquier evolución del sistema hacia un escenario de uso más amplio.

5. Conclusiones y trabajo futuro

[placeholder]

5.1. Servicio de ingesta de telemetría

El servicio de ingesta es el único componente cuyo fallo podría provocar pérdida irrecuperable de información. Su implementación se ha diseñado con tres principios: mínima latencia, robustez frente a pérdida de paquetes UDP y ausencia de dependencias innecesarias.

5.1.1. Bucle principal y gestión del socket UDP

El núcleo es un único hilo que mantiene un socket UDP bloqueante en el puerto configurado y procesa los paquetes en orden de llegada. Cada paquete se lee como buffer de bytes, se inspecciona su cabecera de 29 bytes para identificar el tipo, y se despacha al decodificador correspondiente según el campo packetId. Un único hilo bloqueante es preferible a un modelo asíncrono o multihilo: a 20 Hz el tiempo medio entre paquetes es de 50 ms, muy superior al consumido por cada decodificación, y la complejidad adicional de la concurrencia abriría posibilidades de errores sutiles sin beneficio observable.

El tamaño del buffer de recepción del socket se ha incrementado respecto al valor por defecto del sistema operativo, para absorber ráfagas durante operaciones puntuales del proceso (escrituras a la base, pausas del recolector de basura) sin que el kernel descarte paquetes por desbordamiento.

5.1.2. Decodificación de los paquetes

De los 16 tipos de paquete que define la especificación (Electronic Arts, 2025), el sistema utiliza 7: sesión, datos por vuelta, eventos, participantes, estado del coche, daños del coche e historial de sesión. El resto se descartan en la capa de despacho. Cada decodificador se ha implementado como una secuencia lineal sobre ByteBuffer configurado en little endian, sin recurrir a bibliotecas externas de generación automática: el código resultante es modesto y mantener la lógica explícita prima sobre la sofisticación.

5.1.3. Detección de sectores completados y persistencia de instantáneas

El servicio mantiene en memoria una estructura de estado por cada uno de los 20 vehículos. Cuando el sector actual cambia respecto al almacenado, interpreta el evento como un sector

completado y construye una instantánea completa con información procedente de cuatro paquetes distintos (datos por vuelta, estado del coche, daños y participantes), todos refrescados continuamente sobre el estado en memoria. Este diseño evita tener que sincronizar la llegada de los distintos tipos de paquete.

La instantánea se persiste mediante una sentencia preparada sobre `sector_snapshots`, ejecutada en el mismo hilo del bucle principal. La latencia media de una inserción contra Oracle AI Database 26ai en la misma máquina es de unos pocos milisegundos, muy por debajo del margen disponible entre paquetes consecutivos, por lo que un modelo asíncrono no se justifica.

5.1.4. Estrategia de recuperación frente a pérdida de paquetes UDP

UDP no garantiza la entrega ni el orden, por lo que la pérdida del paquete específico en el que un coche completa un sector se traduce en la ausencia de la instantánea correspondiente. El servicio implementa una estrategia de recuperación en tres niveles complementaria a la captura primaria.

El primer nivel se apoya en los tiempos acumulados que el paquete de datos por vuelta sigue transmitiendo después del sector completado: aunque el paquete específico del cambio de sector se haya perdido, el siguiente paquete contendrá el nuevo número de sector y los tiempos acumulados anteriores, permitiendo reconstruir la instantánea. El segundo nivel utiliza el paquete de historial de sesión, que contiene una versión validada de los tiempos por sector y se emite con menor frecuencia. El servicio mantiene un buffer circular de las cinco vueltas más recientes por vehículo como segunda fuente de relleno. El tercer nivel se utiliza solo cuando los dos anteriores fallan: marca la fila como recuperación incompleta para que la calibración y la simulación puedan filtrar o ponderar los datos de menor calidad. El campo `recovery_source` indica el nivel del que procede cada fila.

5.2. Esquema de la base de datos y procedimientos de persistencia

El esquema relacional del sistema, cuya estructura general se ha presentado en el capítulo anterior, contiene nueve tablas principales y varias secundarias destinadas al almacenamiento de los resultados intermedios de la simulación. Las decisiones de diseño más relevantes, desnormalización selectiva en `sector_snapshots`, clave primaria natural compuesta, ausencia de tablas hijas para los datos de neumáticos y daños, se discutieron en el capítulo dedicado a

la arquitectura. En esta sección se describen los aspectos relativos a los procedimientos de persistencia y a las consultas que sostienen el resto del sistema.

Las inserciones sobre `sector_snapshots` se ejecutan como sentencias preparadas individuales, sin agrupar por lotes, dado que el patrón temporal de los sectores completados dispersa naturalmente las inserciones a lo largo de la duración de la sesión y no se acumulan ráfagas que justifiquen el procesamiento por lotes. Las consultas que el módulo de calibración ejecuta sobre la misma tabla son, por el contrario, fundamentalmente analíticas: agregaciones por compuesto de neumático y por edad, regresiones lineales sobre el tiempo por sector frente a variables explicativas y filtros por circuito y por régimen físico. Estas consultas se benefician de los índices secundarios que el esquema define sobre las columnas más utilizadas en las cláusulas de filtrado y de agrupación.

El modelo aprovecha varias capacidades específicas de Oracle AI Database 26ai, entre ellas el soporte transaccional sobre operaciones de mensajería, que permite combinar la inserción de una fila y la emisión del mensaje correspondiente en la misma transacción, la disponibilidad de tipos numéricos de precisión arbitraria para las columnas que almacenan tiempos en milisegundos sin pérdida de información y la integración con las colas `TxEventQ` que se discute en una sección posterior.

5.3. Módulo de calibración estadística

El módulo de calibración es el componente que traduce los datos brutos acumulados en las sesiones del simulador en un modelo cuantitativo del comportamiento físico del juego, utilizable por el motor de simulación Monte Carlo. Su implementación se ha realizado en Python 3.12 como una herramienta autónoma de línea de comandos que se invoca al término de cada sesión, ya sea automáticamente, como respuesta a un mensaje `TxEventQ` emitido por el servicio de ingesta, o manualmente desde el portal web, y que produce como salida un nuevo conjunto de coeficientes que se persiste en la base de datos.

5.3.1. Modelo matemático del tiempo por sector

El modelo expresa el tiempo por sector como una combinación aditiva de términos asociados a fenómenos físicos identificables:

$$t_{sector} = t_{base} + \Delta_{tyre} + \Delta_{fuel} + \Delta_{damage} + \Delta_{dirty_air} + \Delta_{DRS} + \varepsilon$$

donde t_{base} es el tiempo base característico del piloto y del sector, Δ_{tyre} depende del compuesto y la edad del neumático, Δ_{fuel} varía linealmente con el combustible restante, Δ_{damage} recoge el daño aerodinámico acumulado, $\Delta_{(dirty_air)}$ actúa por debajo de un umbral de proximidad al coche anterior, Δ_{DRS} se activa en los sectores donde está habilitado, y ϵ es el residuo no explicado por el modelo.

La formulación aditiva se ha elegido por la simplicidad del ajuste y la facilidad de interpretación de los coeficientes. Una formulación multiplicativa sería teóricamente más fiel a la naturaleza física de algunos fenómenos, la pérdida de carga aerodinámica por aire sucio es genuinamente proporcional al rendimiento base, pero exige un procedimiento de ajuste no lineal más complejo. La transición a una formulación multiplicativa, si los residuos del modelo muestran un patrón sistemático correlacionado con el tiempo base, se identifica en el Capítulo 7 como línea de trabajo futuro.

5.3.2. Procedimiento de ajuste y separación de regímenes

El procedimiento se ejecuta sobre las filas de `sector_snapshots` del circuito en cuestión. Antes de la regresión, los datos se filtran: se excluyen las filas marcadas como recuperación incompleta, las vueltas inválidas según el simulador, las primeras vueltas de cada sesión (dominadas por la salida) y las vueltas afectadas por coche de seguridad o coche de seguridad virtual.

A continuación, las filas restantes se dividen en dos subconjuntos disjuntos: jugador (identificado por `playerCarIndex` en la cabecera) y oponentes IA. Sobre cada subconjunto se ejecuta el procedimiento de ajuste independientemente, produciendo dos conjuntos disjuntos de coeficientes.

Para cada subconjunto, el ajuste procede en varias etapas: (1) estimación del tiempo base por sector y piloto como la mediana de los tiempos observados tras descontar el resto de términos; (2) regresión lineal de la degradación del neumático sobre la edad, separada por compuesto; (3) regresión lineal del efecto del combustible sobre el peso restante; (4) ajuste análogo del daño aerodinámico, del aire sucio y del DRS; (5) cálculo del coeficiente de determinación del modelo completo, almacenado junto con los coeficientes como medida de la calidad del ajuste.

5.3.3. Detección de valores atípicos

Antes de la regresión, el procedimiento aplica una detección de valores atípicos basada en el rango intercuartílico (Tukey, 1977) por piloto y por sector. Las filas cuyo tiempo queda fuera del rango $[Q_1 - 1,5 \cdot IQR, Q_3 + 1,5 \cdot IQR]$ se marcan como atípicas y se excluyen. El IQR se prefiere a métodos basados en el supuesto de normalidad por su robustez frente a la presencia de los propios valores atípicos en la distribución que los define.

5.3.4. Persistencia y versionado de los coeficientes

Los coeficientes se persisten en `calibration_coefficients`, con clave subrogada y columnas para el circuito, el régimen físico, la versión, la fecha, el coeficiente de determinación y el tamaño muestral. La política de versionado nunca sobrescribe coeficientes existentes: cada ejecución añade una nueva versión, permitiendo reconstruir a posteriori el estado de la calibración en cualquier momento y comparar la evolución del modelo a medida que se acumulan datos.

5.4. Motor de simulación Monte Carlo

El motor de simulación Monte Carlo es el componente computacionalmente más intensivo del sistema. Su tarea consiste en, dado el estado actual de una carrera y un conjunto de estrategias candidatas, ejecutar varias miles de simulaciones independientes para cada estrategia, agregar los resultados en distribuciones de probabilidad sobre las posiciones finales y devolver al usuario una caracterización completa del comportamiento esperado de cada candidata. La implementación se ha realizado en Python con FastAPI (Ramírez, 2025) como capa de exposición HTTP y como consumidor del mensaje TxEventQ que dispara cada ejecución.

5.4.1. Estructura interna de una iteración

Cada iteración Monte Carlo simula la totalidad de los sectores restantes vehículo por vehículo y sector por sector. El estado mantenido por la iteración incluye, para cada uno de los 20 coches: posición, compuesto y edad del neumático, combustible restante, daño aerodinámico acumulado, gap al coche precedente y elegibilidad del DRS. En cada paso, el motor calcula el tiempo del sector aplicando la formulación del modelo y añade un término de ruido residual extraído de una distribución normal con varianza igual a la varianza residual de la calibración (Rubinstein y Kroese, 2017).

Una vez calculados los tiempos de todos los vehículos para un sector, el motor resuelve las interacciones entre coches adyacentes: actualiza gaps, decide adelantamientos según diferencias de tiempo y disponibilidad del DRS, propaga el efecto del aire sucio sobre los coches que terminan siguiendo a otro a corta distancia y registra los cambios de posición. Tras procesar todos los sectores restantes, la iteración produce un orden final de los 20 vehículos.

5.4.2. Generación y poda de estrategias candidatas

Las estrategias candidatas que el motor evalúa se generan automáticamente a partir del estado actual de la carrera mediante un módulo específico cuya tarea es producir un conjunto manejable de alternativas razonables. La generación contempla tres tipos de estrategias: estrategias de cero paradas, estrategias de una parada y estrategias de dos paradas. Las estrategias de cero paradas solo se consideran cuando el reglamento de las dos compuestos obligatorios ya ha sido satisfecho por una parada anterior y la vida útil del neumático en uso supera el número de vueltas restantes. Las estrategias de una parada varían el momento de la parada en pasos de unas pocas vueltas y combinan cada momento con cada compuesto disponible. Las estrategias de dos paradas se consideran únicamente cuando quedan al menos quince vueltas, y se construyen como producto cartesiano de los compuestos disponibles para ambas paradas, con ventanas de pit stops escalonadas y una longitud mínima de relevo de cinco vueltas.

Una vez generadas, las estrategias candidatas se filtran para eliminar las que claramente no son competitivas: las que utilizan compuestos cuyo retraso por vuelta respecto al neumático en uso supera un umbral configurado, las que violan el reglamento de los dos compuestos y, si tras este filtrado el número total de candidatas excede un máximo configurable, las menos prometedoras se podan dando prioridad a las estrategias de cero y una parada sobre las de dos paradas.

5.4.3. Agregación de resultados y producción de la respuesta

Tras ejecutar el número configurado de iteraciones para cada estrategia candidata, los resultados se agregan en distribuciones de probabilidad sobre las posiciones finales de cada vehículo. La estrategia se caracteriza por la posición media esperada, el intervalo de confianza al noventa y cinco por ciento, la probabilidad de finalizar entre los tres primeros, la probabilidad de no finalizar la carrera y el número total esperado de puntos según el sistema

de puntuación de la categoría. Las estrategias se ordenan por el número esperado de puntos y se devuelven al usuario en formato JSON a través del backend.

5.5. Capa de coordinación mediante TxEventQ

La capa de coordinación entre componentes se asienta sobre las colas transaccionales TxEventQ nativas de Oracle AI Database 26ai (Oracle Corporation, 2025b). El sistema utiliza tres colas distintas: una para los eventos de finalización de sesión que disparan la calibración, una para las peticiones de simulación que el motor consume y una tercera para las notificaciones de finalización de simulación que el backend escucha para devolver los resultados a los clientes. Las tres colas se han creado con persistencia transaccional, lo que garantiza que los mensajes sobreviven a un reinicio del servidor de base de datos y que las operaciones de encolado y desencolado pueden participar en transacciones más amplias.

El consumo de los mensajes desde los servicios Python se realiza mediante la biblioteca oficial `python-oracledb`, que proporciona un cliente con soporte nativo para las operaciones de cola. El consumo es bloqueante con un tiempo máximo de espera configurable, lo que permite que los servicios consumidores funcionen como procesos de larga duración sin necesidad de polling activo ni de mecanismos auxiliares de notificación.

5.6. Backend, portal web y cliente iOS

El backend se ha implementado en Spring Boot 3.5.3 (VMware, 2025) y expone dos tipos de interfaz hacia los clientes. La interfaz REST cubre las operaciones de consulta histórica, listado de sesiones, detalle de una sesión concreta, consulta de los coeficientes de calibración vigentes, y las operaciones que disparan ejecuciones diferidas como el lanzamiento manual de una calibración o de una simulación. La interfaz WebSocket cubre las actualizaciones en tiempo real durante el desarrollo de la carrera, en las que el backend reenvía a los clientes conectados los datos en directo que recibe del servicio de ingesta a través de la conexión TCP.

El portal web Angular ofrece una vista de cuadro de mandos en directo con las posiciones, los gaps, los tiempos y el estado de los neumáticos del conjunto de la parrilla, una vista de comparación entre estrategias candidatas que muestra gráficamente las distribuciones de probabilidad y las métricas resumen, una vista de gestión del estado de la calibración que permite consultar las versiones disponibles y disparar nuevas ejecuciones, y una vista de exploración del historial de sesiones almacenadas en la base de datos. La aplicación utiliza el

sistema de signals introducido en versiones recientes de Angular (Google, 2025) para la gestión reactiva del estado del cliente, lo que simplifica considerablemente el código asociado a las actualizaciones en directo.

El cliente iOS, desarrollado en SwiftUI, está específicamente concebido para acompañar al piloto durante el desarrollo de la carrera. Su única función relevante desde el punto de vista del usuario es la reproducción por voz de los mensajes generados por el ingeniero de carrera virtual conforme van produciéndose. La aplicación abre una conexión WebSocket contra el backend al iniciar la sesión, recibe los mensajes serializados como objetos JSON y los pasa al motor de síntesis de voz del sistema operativo a través del API AVSpeechSynthesizer (Apple Inc., s.f.). Los mensajes incluyen un nivel de prioridad que el cliente utiliza para decidir si interrumpir la reproducción de un mensaje en curso o esperar a que termine; los mensajes de prioridad alta, avisos de bandera, instrucciones inmediatas de parada en boxes, interrumpen cualquier mensaje en curso, mientras que los mensajes informativos esperan su turno.

5.7. Capa de generación de mensajes mediante modelo de lenguaje

La capa de generación traduce los resultados cuantitativos del simulador y los eventos discretos detectados durante la carrera en mensajes en lenguaje natural que el cliente iOS reproduce al piloto. La implementación sigue el patrón de generación condicional con plantillas: el modelo de lenguaje no produce contenido libre, sino que selecciona y rellena plantillas predefinidas de un catálogo cerrado.

El catálogo cubre las situaciones recurrentes de una carrera: instrucciones de pit stop, informaciones de gap respecto a rivales, advertencias de tráfico, comunicaciones meteorológicas, avisos de DRS, recomendaciones de gestión de neumáticos y mensajes de encuadre estratégico. Cada plantilla está parametrizada por variables que el motor de simulación o los detectores de eventos rellenan con los valores del momento.

La intervención del LLM consiste en seleccionar la plantilla adecuada y generar pequeñas variaciones léxicas que reduzcan la sensación de repetición. La selección se restringe a un conjunto cerrado y validado, lo que limita el riesgo de contenido inadecuado y mantiene la salida dentro de un margen predecible. Esta arquitectura conservadora se inscribe en las recomendaciones de la literatura reciente sobre agentes basados en LLMs para entornos donde la fiabilidad de la salida es prioritaria (Capítulo 2).

El LLM se invoca por HTTP, con una latencia de varios cientos de milisegundos por mensaje. Esta latencia es compatible con el ritmo de los eventos estratégicos relevantes durante una carrera y se mide como una componente de la latencia total del sistema en el Capítulo 6.

6. Evaluación y resultados

[placeholder]

6.1. Latencia extremo a extremo del pipeline

[placeholder]

6.2. Convergencia del motor de simulación Monte Carlo

[placeholder]

6.3. Ajuste estadístico del modelo de calibración

[placeholder]

6.4. Valoración cualitativa de los mensajes de voz generados

[placeholder]

6.5. Discusión global de los resultados

[placeholder]

7. Conclusiones y trabajo futuro

7.1. Conclusiones del trabajo

[placeholder]

7.2. Lineas de trabajo futuro

[placeholder]

Referencias bibliográficas

- Aguad, F., y Thraves, C. (2024). Optimizing pit stop strategies in Formula 1 with dynamic programming and game theory. *European Journal of Operational Research*, 319(3), 908–919. <https://doi.org/10.1016/j.ejor.2024.06.038>
- Apple Inc. (s.f.). AVSpeechSynthesizer. Apple Developer Documentation. <https://developer.apple.com/documentation/avfaudio/avspeechsynthesizer>
- Bekker, J., y Lotz, W. (2009). Planning Formula One race strategies using discrete-event simulation. *Journal of the Operational Research Society*, 60(7), 952–961. <https://doi.org/10.1057/palgrave.jors.2602626>
- Brawn, R., y Parr, A. (2016). *Total competition: Lessons in strategy from Formula One*. Simon & Schuster.
- Carrero, Z. I., y cols. (2026). Benchmarking large language model-based agent systems for clinical decision tasks. *npj Digital Medicine*, 9, Article 26. <https://doi.org/10.1038/s41746-026-02443-6>
- Doran, G. T. (1981). There's a S.M.A.R.T. way to write management's goals and objectives. *Management Review*, 70(11), 35–36.
- Electronic Arts. (2025). EA Sports F1 25 UDP specification. <https://forums.ea.com/blog/f1-games-game-info-hub-en/ea-sports-f1-25-udp-specification/12187347>
- Google. (2025). Angular signals guide (Version 21). <https://angular.dev/guide/signals>
- Heilmeyer, A., Graf, M., y Lienkamp, M. (2018). A race simulation for strategy decisions in circuit motorsports. En *2018 21st International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2997–3004). IEEE. <https://doi.org/10.1109/ITSC.2018.8570012>
- Heilmeyer, A., Graf, M., Betz, J., y Lienkamp, M. (2020). Application of Monte Carlo methods to consider probabilistic effects in a race simulation for circuit motorsport. *Applied Sciences*, 10(12), Article 4229. <https://doi.org/10.3390/app10124229>
- Hojaji, F., Smithies, T. D., Kennedy, J., Toth, A. J., y Campbell, M. J. (2024). AI-enabled prediction of sim racing performance using telemetry data. *Computers in Human Behavior Reports*, 14, Article 100414. <https://doi.org/10.1016/j.chbr.2024.100414>

- Lin, Y., y cols. (2025). From language to action: A review of large language models as autonomous agents and tool users. arXiv preprint. <https://arxiv.org/abs/2508.17281>
- Liu, X., y Fotouhi, A. (2020). Formula-E race strategy development using artificial neural networks and Monte Carlo tree search. *Neural Computing and Applications*, 32(18), 15191–15207. <https://doi.org/10.1007/s00521-020-04871-1>
- Metropolis, N., y Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335–341. <https://doi.org/10.1080/01621459.1949.10483310>
- Oracle Corporation. (2024). Java SE 23 & JDK 23 API specification. <https://docs.oracle.com/en/java/javase/23/docs/api/>
- Oracle Corporation. (2025a). Oracle AI Database 26ai new features guide. <https://docs.oracle.com/en/database/oracle/oracle-database/26/nfcoa/>
- Oracle Corporation. (2025b). Transactional event queues and advanced queuing user's guide (Release 26ai). <https://docs.oracle.com/en/database/oracle/oracle-database/23/adque/>
- Ramírez, S. (2025). FastAPI (Version 0.115). <https://fastapi.tiangolo.com/>
- Rubinstein, R. Y., y Kroese, D. P. (2017). *Simulation and the Monte Carlo method* (3.^a ed.). Wiley. <https://doi.org/10.1002/9781118631980>
- Thomas, D., Jiang, J., Kori, A., Russo, A., Winkler, S., Sale, S., McMillan, J., Belardinelli, F., y Rago, A. (2025). Explainable reinforcement learning for Formula One race strategy. En *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing* (pp. 1090–1097). ACM. <https://doi.org/10.1145/3672608.3707773>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- VMware. (2025). Spring Boot reference documentation (Version 3.5.3). <https://docs.spring.io/spring-boot/docs/3.5.3/reference/htmlsingle/>

Anexo A. Título del anexo.

[placeholder]

Índice de acrónimos

Este índice es opcional, y no es necesario en la mayoría de los trabajos. Si decides incluir un índice de acrónimos recuerda que estos deben ir ordenados por índice alfabético.